

COMPLETE FUNCTION CHEAT SHEET

Domain used here:

"University Operations & Analytics System"

Use this file to:

- Understand logic patterns
- Avoid common mistakes
- Prepare for assessments confidently

"""

```
import pandas as pd
```

```
import numpy as np
```

```
# =====
```

```
# SECTION 1: OOP – DICTIONARY STATE MANAGEMENT
```

```
# =====
```

```
class UniversityRegistry:
```

```
    """
```

```
    def __init__(self):
```

```
        self.data = {}
```

```
    # --- Add / Register Pattern ---
```

```
    def add_entity(self, key: str, value: int) -> dict:
```

```
        """
```

```
        Pattern used in:
```

- add_intersection
- register_participant
- add_product

```
        """
```

```
        if key in self.data:
```

```
            self.data[key] += value
```

```
        else:
```

```
            self.data[key] = value
```

```
        return self.data
```

```
    # --- Update / Fulfill / Cancel Pattern ---
```

```
    def reduce_entity(self, key: str, value: int) -> dict:
```

```
        if key not in self.data:
```

```

        raise ValueError("Key not found")

    if value > self.data[key]:
        raise ValueError("Insufficient quantity")

    self.data[key] -= value
    return self.data

# --- Reschedule / Merge Pattern ---
def merge_entities(self, old_key: str, new_key: str) -> dict:

    if old_key not in self.data:
        return self.data

    count = self.data[old_key]

    if new_key in self.data:
        self.data[new_key] += count
    else:
        self.data[new_key] = count

    del self.data[old_key]
    return self.data

# --- Filter Active Entities ---
def list_active(self) -> list:

    return [k for k, v in self.data.items() if v > 0]

# =====
# SECTION 2: DATAFRAME CREATION (FOUNDATIONAL)
# =====

class UniversityAnalytics:
    """
    Covers all Pandas-based assessments.
    """

    def create_df(self, rows: list, columns: list) -> pd.DataFrame:

        return pd.DataFrame(rows, columns=columns)

# =====
# SECTION 3: GROUPBY + AGGREGATION
# =====

```

```
def group_sum_mean(self, df: pd.DataFrame, group_col: str, value_col: str) ->
pd.DataFrame:
```

```
    result = (
        df.groupby(group_col)[value_col]
        .agg(["sum", "mean"])
        .reset_index()
    )
    return result
```

```
# =====
# SECTION 4: RATE / PERCENTAGE CALCULATION
# =====
```

```
def compute_rate(self, df: pd.DataFrame, group_col: str, flag_col: str, flag_value) ->
pd.DataFrame:
```

```
    total = df.groupby(group_col).size()
    filtered = df[df[flag_col] == flag_value].groupby(group_col).size()

    rate = (filtered / total * 100).fillna(0).round(1)
    return rate.reset_index(name="Rate")
```

```
# =====
# SECTION 5: ADD DERIVED COLUMNS
# =====
```

```
def add_boolean_flag(self, df: pd.DataFrame, col: str, threshold: float) -> pd.DataFrame:
    """
```

```
    df = df.copy()
    df["Flag"] = df[col] > threshold
    return df
```

```
def add_mapped_points(self, df: pd.DataFrame, source_col: str) -> pd.DataFrame:
    """
```

```
    Pattern used in:
    - add_result_points
    """

    mapping = {"High": 3, "Medium": 1, "Low": 0}
    df = df.copy()
    df["Points"] = df[source_col].map(mapping)
    return df
```

```
# =====
# SECTION 6: FILTERING
# =====
```

```
def filter_rows(self, df: pd.DataFrame, col: str, threshold: float) -> pd.DataFrame:
```

```
    return df[df[col] > threshold]
```

```
# =====
```

```
# SECTION 7: SORT + TOP-N
```

```
# =====
```

```
def top_n(self, df: pd.DataFrame, col: str, n: int) -> pd.DataFrame:
```

```
    return df.sort_values(by=col, ascending=False).head(n)
```

```
# =====
```

```
# SECTION 8: CLEANING
```

```
# =====
```

```
def clean_and_sort(self, df: pd.DataFrame, sort_col: str) -> pd.DataFrame:
```

```
    df = df.dropna()
```

```
    return df.sort_values(by=sort_col, ascending=False)
```

```
# =====
```

```
# SECTION 9: NUMPY – ARRAY VALIDATION & ANALYTICS
```

```
# =====
```

```
class NumericAnalyzer:
```

```
    def validate_array(self, arr: np.ndarray) -> bool:
```

```
        """
```

```
        Pattern used in:
```

```
        - validate_array
```

```
        """
```

```
        arr = np.asarray(arr)
```

```
        if arr.size == 0:
```

```
            return False
```

```
        if not np.issubdtype(arr.dtype, np.number):
```

```
            return False
```

```
        if np.any(np.isnan(arr)):
```

```
            return False
```

```
        if np.any((arr < 0) | (arr > 100)):
```

```
            return False
```

```
return True
```

```
def sum_array(self, arr: np.ndarray) -> float:
```

```
    """
```

```
    Pattern used in:
```

```
    - sum_array
```

```
    """
```

```
    return float(np.sum(arr))
```

```
def mean_array(self, arr: np.ndarray) -> float:
```

```
    """
```

```
    Pattern used in:
```

```
    - mean_array
```

```
    """
```

```
    return float(np.mean(arr))
```

```
def normalize_array(self, arr: np.ndarray) -> np.ndarray:
```

```
    """
```

```
    Pattern used in:
```

```
    - normalize_array
```

```
    """
```

```
    arr = np.asarray(arr, dtype=float)
```

```
    return (arr - np.mean(arr)) / np.std(arr)
```

```
def count_above_threshold(self, arr: np.ndarray, threshold: float) -> int:
```

```
    """
```

```
    Pattern used in:
```

```
    - count_above_threshold
```

```
    """
```

```
    return int(np.sum(arr > threshold))
```

```
# =====
```

```
# SECTION 10: COMMON STUDENT MISTAKES (VERY IMPORTANT)
```

```
# =====
```

```
    """
```

```
DO NOT DO THESE:
```

```
-----
```

```
- return {}
```

```
- return []
```

```
- return fixed DataFrames
```

```
- hardcode example outputs
```

```
- use print() inside solution
```

```
- use .head(3) when n is parameterized
```

```
- skip df.copy()
```

```
- ignore missing keys
```

```
- ignore ValueError conditions
```

- modify input objects directly

ALWAYS DO:

- Use groupby + reset_index
- Validate inputs
- Handle empty / missing cases
- Return correct type & structure
- Match column names EXACTLY
- Follow LLD word-by-word

""""

```
# =====  
# END OF COMPLETE MASTER CHEAT SHEET  
# =====
```